XiaoCai AI 小财 AI — Feature Guide

The Business Profit & Survival Agent · SME Finance Copilot

"See what's really left — before it's too late." · **Data in. Math decides. AI explains.**

Shenzhen FinTechathon 2026 · International Track · Topic E | Document generated 2026-10-19 | Status: **Phase 1 of 8 complete**

13 DB models (money in fen)	**4+1** API endpoints live	**5** web routes live + 9 nav stubs	**2** languages (EN / 中文)	**7/7** unit tests green	**6** commits, clean history

1 · Product in one page

Multi-channel sellers in China (Douyin, Taobao/Tmall, Xiaohongshu, Pinduoduo) see record sales but an empty bank account: fees, refunds, ads, influencer commissions, delivery and D15–D28 settlement delays hide the real cash position. It is a **visibility problem, not a sales problem**. Anchor example: ¥100,000 revenue → product 25k + ads 25k + commission 20k + delivery 12k + refunds 8k + platform fees 5k → **real profit ¥5,000 (5%)**.

Owner question	Engine	Delivers	Status
"How much do I really keep?"	True Profit Detective	Profit per product / campaign / channel + cost waterfall	Phase 2–6
"Why did profit drop?"	Driver Explainer	Period-over-period variance decomposed, ranked, with evidence	Phase 2–6
"Will I run short of cash?"	Survival Simulator	30-day cash curve, what-if sliders, shortage day, alerts	Phase 2–6

Non-negotiable architecture rule: authorised data → deterministic engine (the *only* source of numbers) → AI layer that parses questions & writes explanations (post-checked: any number not in the engine JSON is discarded → template fallback) → owner approves every financial action. The app must fully work with the LLM turned off.

2 · Architecture & monorepo layout

```
xiaocai-ai/  (npm workspaces · Node ≥ 20 · strict TypeScript, no "any")
├─ apps/web        Next.js 14 App Router · Tailwind · shadcn-style UI · next-intl (EN/中文)
├─ apps/api        Express 4 · Zod · Prisma 5.22 · JWT httpOnly cookie · pino · helmet
│   ├─ prisma/     schema.prisma · migrations/0_init · (seed.ts in Phase 3)
│   ├─ src/routes/ health · auth            (Phase 4+: profit, cash, financing, …)
│   ├─ src/lib/    prisma(443 auto-driver) · jwt · password · cookies · logger · async
│   ├─ src/middleware/ auth(requireAuth/requireRole) · rate-limit · error
│   ├─ src/audit/  writeAudit — append-only, fire-and-forget, no PII
│   └─ scripts/    db-check · db-migrate-443 · embedded-pg · neon-probe · e2e-audit-and-cleanup
├─ packages/engine pure TS — no I/O, no LLM, no clock. Money = integer fen. (Phase 2 fills it)
└─ docs/           this guide
```

🌐 **Network-proof DB path:** this ISP silently drops Postgres payloads on port 5432 (TCP fake-connects). When `DATABASE_URL` host is `*.neon.tech`, the API automatically connects via the Neon WebSocket driver over **HTTPS 443** (`@prisma/adapter-neon`); other hosts use plain TCP. Migration CLI still needs TCP → use `npx tsx apps/api/scripts/db-migrate-443.ts` on this network. Verified live: migrate + register/login against real Neon.

3 · Database — 13 models DONE

All money columns are **Int storing fen** (¥1 = 100); formatted only at display. Spec'd compound indexes: `Order(businessId, orderedAt)`, `AdSpend(businessId, date)`, `SupplierInvoice(businessId, dueDate)`.

Model	Key fields	Notes
Business	name, nameZh, currency=CNY, openingCash, **isDemo**	demo data flag (Phase 3 seed)
User	email UQ, passwordHash, name, **role: OWNER/ACCOUNTANT/VIEWER**	cascade on business delete
DataSource	type: CHANNEL/ADS/LOGISTICS/INVOICES/BANK · status: GRANTED/REVOKED · permission: READ_ONLY/READ_WRITE · grantedAt/revokedAt	revoked ⇒ hidden from engines (Phase 3 helper)
Product	sku (UQ per business), name, unitCost	—
Order	channel+externalId (UQ), orderedAt, gross, platformFee, commission, deliveryCost, refundAmount, status, settlementDate, settledAmount	core profit input
AdSpend	channel, campaign, date, spend, attributedRevenue	ROAS driver
SupplierInvoice	supplierName, amount, dueDate, status: PAID/DUE/OVERDUE/**MISSING_DOC**, documentRef	missing docs disclosed, never hidden
Settlement	channel, expectedDate, amount, PENDING/RECEIVED	D15–D28 delays
Scenario	name, params Json, result Json	saved what-ifs (Phase 4)
Recommendation	type, title, **evidence Json · impact Json · action Json**	Evidence·Impact·Action shape, always
Approval	1:1 recommendation, PENDING/APPROVED/REJECTED, decidedById/At, note	approve ≠ execute — record only
AuditLog	action, entityType/Id, metadata, ip — **insert-only**	hash chain lands Phase 7
ImportJob	kind, filename, rowsTotal/Ok/Failed, errors Json, status	CSV pipeline (Phase 3)

4 · Backend features

4.1 Live endpoints DONE & E2E-TESTED

Endpoint	Behaviour	Verified
`GET /health`	runs SELECT 1 → 200 `{status:"ok",db:true}` / 503 `degraded` when DB unreachable	✓ live
`POST /api/auth/register`	Zod-validated → creates **Business + OWNER user in one transaction** → sets httpOnly cookie → audit REGISTER. 409 duplicate email · 400 validation	✓ 201
`POST /api/auth/login`	rate-limited (10 fails / 15 min, successes don't count) → generic 401 (no user enumeration) → audit LOGIN / LOGIN_FAILED (no emails logged)	✓ 200/401/429
`POST /api/auth/logout`	clears cookie with proper past-dated Expires (bug found & fixed in E2E) → audit LOGOUT	✓ 204
`GET /api/auth/me`	requireAuth → re-fetches user from DB (role changes take effect instantly) → returns user + business, **passwordHash never leaves the server**	✓ 200/401

4.2 Security implemented DONE

- bcrypt (10 rounds) hashing · JWT HS256, 7-day expiry in **httpOnly + SameSite=Lax + Secure-in-prod** cookie
- `requireAuth` / `requireRole(...)` middleware (role guard ready for Phase 4 endpoints)

- helmet (CSP, HSTS, nosniff, frameguard…), strict CORS with credentials (WEB_ORIGIN), 1 MB JSON body cap
- Two rate limiters: global 300/15 min · login 10 fails/15 min (express-rate-limit v7)
- Central error handler: Zod→400 VALIDATION · Prisma P2002→409 · everything else→500 `{error:"INTERNAL"}` with **no stack leakage**; unknown routes → JSON 404
- Append-only AuditLog on REGISTER / LOGIN / LOGIN_FAILED / LOGOUT with IP; cookies/authorization headers redacted in all logs

4.3 Planned backend work

Phase	Feature	Details
2	Deterministic engine	trueProfit (reproduces ¥100k→¥5k) · profitDrivers · cashForecast (Day-12 shortage) · applyScenario · financingMatcher · buildEvidenceTree · dailyBriefData — pure functions, 40+ Vitest tests
3	Seed + CSV import	Ms. Li sandbox (GMV ¥180k, settled ¥142k, supplier ¥90k, 600 orders) · POST /api/imports/:kind with multer+papaparse, header mapping, row-level errors, ImportJob · DataSources grant/revoke → `getAuthorisedData()` becomes the only engine data path
4	Engine REST API	/api/profit/summary|by-product|by-campaign|by-channel|drivers · /api/cash/forecast|scenario|alerts · /api/financing/options · /api/evidence/:metric/:level · /api/brief/daily · /api/recommendations · /api/approvals/:id/decision (OWNER only) · /api/audit(+export.csv) · DATA_ACCESS audit on every financial read · supertest integration tests · swagger /docs
5	AI layer (explain-only)	LlmProvider interface (Claude / Noop fallback) · parseWhatIf (Zod, clamp –80..+200, EN+中文) · explain() with numbers-must-come-from-engine post-check · template explanations · prompt-injection guard (CSV cells never reach the LLM)
7	Hardening	CSRF double-submit token · account lockout · password policy · audit hash chain (prevHash+hash) · buyer-ID hashing (no PII) · npm audit + GitHub Actions CI · docs/ (SECURITY_SELF_ASSESSMENT, DATA_FLOW, RISK_REGISTER, PRIVACY_NOTES)
8	Ship	Playwright e2e ×2 · Vercel + Render/Railway + Neon deploy · README · /submission pack · DEMO_SCRIPT.md

5 · Frontend features

5.1 Live routes DONE

Route	What's there
`/`	Landing hero — navy band, tagline, Register/Log-in CTAs, phase badge
`/register`	Name, business name (EN + 中文 optional), email, password (min 8) → auto-login → dashboard
`/login`	Email + password; friendly 401 copy; links both ways with register
`/dashboard`	Header row with **live API-status chip** (green/red/gray, fetches /health) · 4 KPI placeholder cards (Revenue, Real Profit, Cash Today, Days-to-Shortage) marked "Lands in Phase 6"

5.2 App shell & design system DONE

- **Navy + emerald + warm-amber tokens** (red reserved for shortage only) via CSS variables + Tailwind config; shadcn-style primitives: Button (6 variants), Input, Label, Card set
- **Sidebar** with all 10 destinations: Dashboard ✅ + True Profit, Why Profit Dropped, Survival Simulator, Financing, Daily Brief, Approvals, Data Sources, Audit Log, Security (amber *Soon* badges; active-state highlight)
- **Header**: EN/中文 switcher (cookie-based, `router.refresh()`, aria-pressed) + Logout button
- **Full internationalisation** via next-intl without URL prefixes — `src/i18n/request.ts` reads the `xc_locale` cookie server-side; messages in `src/messages/en.json` + `zh.json`; every string already translated, including auth errors
- **Route guard** `src/middleware.ts`: no session cookie → /dashboard redirects to /login (with `?next=`); logged-in visits to /login|/register → /dashboard (presence check; real verification stays in requireAuth)
- API client `lib/api.ts`: JSON wrapper, credentials:include, typed ApiError; `NEXT_PUBLIC_API_URL` with localhost:4000 fallback

5.3 Planned screens (Phase 6)

Screen	Key UI
Dashboard (full)	4 real KPIs · "Record sales. Empty bank account." GMV-vs-settled-vs-supplier chart · top-3 Evidence·Impact·Action insight cards · alert banner
True Profit	Waterfall chart (revenue→costs→profit) · tabs by product/campaign/channel · sortable table, margin chips
Why Profit Dropped	Period selector · ranked driver bars · click → drill-down drawer
Survival Simulator	Base-vs-stressed cash curve · 4 what-if sliders (debounced live recompute) · red shortage marker · save scenario · chat "If sales drop 20%…" → Day-12 answer + 3 option cards → "Send for my approval"
Financing	4-option table (Cost/Speed/Best-when) · top pick highlighted · disclaimer: qualitative only, owner approves
Tap-any-number	Reusable `<EvidenceNumber>`: every ¥ clickable → 3-level drawer (headline → breakdown → raw order/statement) · amber MISSING_DOC badge "disclosed, not hidden"
Daily Brief	老板每日经营简报 <3 min · 紧急付款/成本上升/逾期发票/待审批/缺失单据 · copy + print
Approvals	Pending list, approve/reject with note, history · banner "XiaoCai never pays or messages anyone on its own"
Audit Log / Security	Filterable table + CSV export · 4 pillars + known-risks R1–R5

6 · DevOps, tooling & run book DONE

Need	Command

Install everything	`npm install`
Run API / Web	`npm run dev:api` (:4000) · `npm run dev:web` (:3000)
Quality gates	`npm run lint` · `npm run typecheck` · `npm test` · `npm run build`
Migrate (normal network)	`npm run db:migrate` (prisma migrate dev)
Migrate on 5432-blocked network	`npx tsx apps/api/scripts/db-migrate-443.ts`
Local DB without Neon	`npx tsx apps/api/scripts/embedded-pg.ts` → postgres://postgres:postgres@localhost:5433/xiaocai
Connection diagnostics	`npx tsx apps/api/scripts/db-check.ts` · `node apps/api/scripts/neon-probe.js <host>`
Browse data	`npm run db:studio`

Env: DATABASE_URL (Neon, in gitignored apps/api/.env only) · JWT_SECRET ≥32 chars · WEB_ORIGIN · PORT · NEXT_PUBLIC_API_URL. Demo login created: demo@xiaocai.ai / Demo1234! (OWNER, "Li's Livestream Store 李女士直播店").

7 · Improvement checklist (work through top-down)

Quick wins you can do today no new phases

- □ Mobile sidebar — currently `hidden md:flex`; add a hamburger drawer (responsive down to 360 px is a Phase-6 requirement)
- □ Register page: live password-strength hint + show confirm-password field
- □ Login page: pre-fill `?next=` redirect target after success (param already sent by middleware)
- □ Dashboard: auto-refresh the API chip every 30 s + show DB-latency from /health
- □ Add favicon + PWA manifest + `next/font` (system stack is fine but Inter would sharpen the fintech look)
- □ Add a "system status" footer line on dashboard: driver in use (443/TCP), DB latency, version

Phase 2 — engine (next)

- □ trueProfit reproduces the ¥100k→¥5k anchor in tests · edge cases: zero revenue, refunds > revenue, empty arrays, negative balances
- □ cashForecast hits Day-12 shortage on the stressed demo inputs (sales −20%, supplier +10%)
- □ financingMatcher stays qualitative — no invented interest rates
- □ 100% of deck-slide numbers (slides 2/3/4/7) covered by unit tests

Phase 3–4 — data & API

- □ Seed tuned to deck numbers, `isDemo=true`, "Illustrative" labels everywhere
- □ `getAuthorisedData(businessId)` as the ONLY raw-table path for engines; revoked sources disappear from every number
- □ CSV import: header mapping + preview + per-row errors + ImportJob + audit; sample templates downloadable
- □ Every financial read writes a DATA_ACCESS audit row · role tests (VIEWER can't approve, ACCOUNTANT can't revoke)

Phase 5 — AI guardrails

- □ Test that proves a fabricated LLM number gets discarded (the guard actually fires)
- □ Chinese + English what-if parsing ("如果销售下降20%…") · confidence clamp −80..+200 · clarifying-question path

Phase 6 — UX bar

- □ Every ¥ becomes an `<EvidenceNumber>` — Level 1→2→3 drawer, raw order refs, MISSING_DOC amber badge
- □ Loading skeletons + empty states + error boundaries on all screens · keyboard/contrast a11y pass
- □ "Try the Ms. Li demo" auto-login button on landing

Phase 7 — security gaps to close (known, by design, tracked)

- □ CSRF double-submit token (SameSite=Lax is the current baseline)
- □ Account lockout + stronger password policy (min-8 today)
- □ Server-side session revocation (logout clears the cookie; a stolen token is valid ≤7d until then)
- □ AuditLog integrity hash chain + immutability trigger in DB
- □ Email enumeration on register (409 reveals existence) — decide policy
- □ Login rate limit is per-instance memory → Redis store if API scales horizontally

Phase 8 — submission

- □ Playwright: login → simulator → "Day 12" → approve → audit shows event · CSV upload → profit updates
- □ Deploy web/api/DB, run migrate deploy + seed server-side · DEMO_SCRIPT.md (8 beats) · /submission pack · verify no secrets in git history

Verification status of everything claimed above

Lint ✓ (3 workspaces) · typecheck ✓ (strict, no any) · 7/7 Vitest ✓ · next build ✓ (5 routes + middleware) · tsc build ✓ · auth E2E on real Neon ✓ (register 201 / me 200 / logout 204 / login 200 / wrong-password 401 / duplicate 409 / validation 400 / rate-limit 429 / audit rows verified in DB) · embedded-pg E2E ✓ · Neon over 443 ✓ (migrate + full auth flow) · git history: zero .env commits (checked programmatically).